

To what extent does scaffolding improve automated theorem provers driven by large agents?

Mikael Bashir - 02382080

14 August 2026

Abstract

We report results from three benchmark runs of the Leak [10] automated theorem-proving stack on FATE-X [13], a 100-problem graduate-level abstract and commutative-algebra benchmark in Lean 4 [19]. The headline finding is negative for our central design hypothesis: on identical time budgets and the same base agent, a single flat agent with compiler and search access (Control II) solved **38/98 (38.8%)**, outperforming the best decomposition pipeline Ultra Fleeting, **28/98, (28.6%)**. Decomposition contributed only two problems the flat agent could not solve. We analyse where each approach wins (§4), quantify a difficulty cliff in FATE-X’s top quartile (2/25 for both arms) (§4.4), mention the events that shaped the design, including the MCP services we made extensive use of (§3.2), and lay out the follow-up experiments the results motivate or that could not be completed due to budget (§10).

Contents

1	Introduction	3
2	Related work	3
3	Experimental setup	4
3.1	Benchmark and normalization	4
3.2	Verification infrastructure	4
3.3	Strategies	5
3.4	Protocol	5
3.5	Metrics	6
4	Head-to-head: decomposition vs. the flat agent	6
4.1	Agreement and discordance	6
4.2	Interpretation	7
4.3	Per-problem economy on shared solves	7
4.4	Difficulty gradient	9
4.5	Solves by topic area	9

5	Case studies	10
5.1	fatex_003: flat-solvable, decomposition-hostile	10
5.2	fatex_061 and fatex_067: where decomposition wins	10
5.3	fatex_083 and fatex_084: the only top-quartile solves	11
6	The base agent alone (Control III)	11
7	Cost analysis	11
8	Operational incidents and data integrity	12
9	Threats to validity	13
10	Conclusions and next steps	13
A	Per-problem results	17

1. Introduction

There has been extensive work on LLMs, specifically on Automated Theorem Proving systems driven by LLMs, with a variety of architecture designs, both in the development of the LLM itself, and the orchestration of the LLM. We are also seeing the number of agentic systems being made publically available increase rapidly, which raises the natural question "Do the same techniques for ATP on LLMs apply to Agentic systems, and to what extent". We present the Leak stack, an agentic theorem-proving system for Lean 4 + Mathlib [19, 24] that tries to answer exactly this, armed with select, specialist remote verification services ("Leak I–XIV") and a local bridge that drives LLM agents against them [9, 16]. Its design bet, shared with a line of recent work on hierarchical and decomposition-based proving [17, 21, 2, 11], is that hard theorems are best attacked by splitting them into sub-goals proved by parallel sub-agents. This report evaluates that bet empirically, while also testing if this still holds in large agentic systems such as Claude, which already have secret scaffolding built into them [3].

We ran several full benchmark passes on a sanitised (§3.1) FATE-X [13, 7], reporting in this paper the results of 3 runs. The experiments were designed to answer three questions:

1. **Absolute proving capacity.** What fraction of an algebra benchmark, exceeding PhD qualifying exam difficulty, can Leak close under fixed time budgets (FATE-X)?
2. **Adding scaffolding vs. Relying on built-in scaffolding.** Does the lifted architecture (blueprint $\rightarrow$ node-provers $\rightarrow$ refinement) beat a single continuous agent given the same model and budget (and select proving tools)?
3. **Does Sonnet 5 benefit significantly when given pantograph?** While we armed many runs with an improved pantograph service [6], we always took for granted its benefit - but in the context of agentic systems, how much does pantograph actually buy.

Questions 1 is objectively answered below, and we provide evidences below for questions 2 and 3.

2. Related work

Automated theorem provers are typically evaluated on miniF2F [27], PutnamBench [25], or competition sets targeted by systems such as GPT-f [20], HyperTree Proof Search [17], AlphaProof [15], DeepSeek-Prover-V2 [21], and the Gödel-Prover line [11]. FATE-X [13] is deliberately harder and narrower: 100 abstract/commutative-algebra problems drawn from graduate textbooks, qualifying exams, and research literature, ordered by increasing difficulty. Its authors report frontier systems solving only single-digit counts on the upper half [7], which matches the cliff we observe (§4.4), though it should be noted that the SOTA models the authors tested with were not used as they were designed (self-correction was disabled).

Decomposition-style pipelines — sketching a proof outline and delegating sub-goals — descend from Draft-Sketch-Prove-like designs and are the core of DeepSeek-Prover-V2’s subgoal decomposition [21] and related hierarchical searches [17, 2]. We test an already researched and documented pipeline, Goedel-Architect [14], changing the driving backbone — instead of a fast and small LLM, such as the DeepSeek-V4-Flash backbone used by Goedel-Architect, we drive with a large agent (Claude Sonnet 5 Agent [4]), referring to this design as Ultra Fleeting through out the paper; as far as we are aware the team behind Goedel Architect have not made available the critical scaffolding on DeepSeek-V4-Flash, so we try to replicate as closely as possible from their report. The main features are a blueprint stage that compiles a skeleton of **sorry**-holes, node provers that fill holes as ReAct-style tool agents [26] against live compiler service feedback, and a refinement stage restructures the skeleton between iterations. Contrary to the paper’s suggestions, we deliberately omit a natural-language seed/proof, on the untested and unverified assumption that recent capability gains — from drivers like DeepSeek-V4-Flash to Sonnet 5 — are concentrated in reasoning rather than knowledge, so seeding a candidate proof should benefit a strong-reasoning driver little in a formal verification task. We also experiment with Pantograph [1, 23], extended with a snapshot-based “ghost state” model [22, 6] that lets many agents manage proof states on a single Lean daemon, without expensive re-elaboration. This service was used for Control-II runs and other experimental runs not presented in this report, but it wasn’t used for the Ultra Fleeting benchmark, in line with the design of the borrowed architecture. In the further-works section (§10), we discuss some interesting experiments that could not be completed before the writing of this report.

3. Experimental setup

3.1 Benchmark and normalization

FATE-X contains 100 Lean files pinning Lean v4.28.0 [12]. Our ingestion (`build-fatex.mjs`) normalizes each item: imports and the `namespace ProblemN` wrapper are dropped, doc comments become the informal statement, `section` scoping is kept, and the 38 problems with supporting declarations get their declarations compressed into the theorem statement (so that problems are in the shape our harness expects). Problems are enqueued in the benchmark’s own difficulty order, so the item index doubles as a difficulty rank. Two statements (`fatex_009`, `fatex_041`) do not elaborate under our newer toolchains and were excluded from the full passes, leaving $n = 98$.

3.2 Verification infrastructure

All scoring is by independent re-verification, never by agent self-report. Two verifier groups were in play, and they differ by toolchain:

- **Leak IV group** (Lean + Mathlib v4.29.1): gates every control run. A daemon compiles a submitted script, and a proof is only ever declared valid if Leak-IV returns no errors or warnings (a proof containing a `sorry` will be caught by our harness), AND the script passes strict regex checks to ensure the same theorem was proven (again enforced by our harness).

- **Leak XI/XII/XIV group** (Lean + Mathlib v4.32.0): serves the architect pipelines (Ultra) — XII compiles and elaborates a proposed blueprint/refinement, XIV verifies full scripts like IV, and XI provides Loogle/Moogle-style search [8, 18] like I.

We have two families of services because of toolchain differences between some of the libraries involved (pantograph and lean architect). It should not be difficult to unify the two families to run on one toolchain, but we opted to keep them separate and focus on the experiments instead, since our experiments did not require pantograph and architect at the same time. All services follow the MCP protocol on SSE transport [5] (future work should consider using Streamable HTTP).

3.3 Strategies

It is important to make clear that all of our strategy designs had their internet search tool stripped (from Claude’s default tool set), so that we could focus on proving capacity, not retrieval capacity. Of course for formal verification, one must be familiar with the syntax, and thus we make available a verbose compiler daemon and toolchain pinned library search tools.

Ultra (Lifted Goedel Architect). An inspired pipeline driven by a local Claude Sonnet 5 CLI [4] instance; **ultra-fleeting** has no recollection of proved lemmas between two separate proof attempts and was not given NL proofs to shape decomposition. This is the only reported run armed with Leak XI, Leak XII, and Leak XIV.

Control II (compiler + lemma search). Same driving agent as Ultra, armed with Leak IV and Leak I [8, 18]; exactly how those tools are used is left to Claude’s built-in harness [3]. Control-II gets no further scaffolding - benchmarked as **control-oneshot-2**. Every control run is allowed built-in Claude Code tools, such as bash, read, write, except internet search.

Control III (blind prover). Same driving agent as Ultra, given *no* additional Leak tools: it is repeatedly asked to prove the target theorem and told only whether each attempt was correct — never an error message — with no context beyond its own previous attempts and the attempt number. A deterministic Leak IV gate decides correctness. Benchmarked as **control-oneshot-3**. This run was designed to test the core agent’s proving capability, and get an absolute baseline.

Control IV (compiler + pantograph + lemma search). Same driving agent as Ultra armed with Leak I, Leak II, and Leak IV, again leaving all orchestration to Claude’s built-in harness [3]. This run was specifically designed to see if a pantograph service improves **overall** proving capacity (more theorems proven).

3.4 Protocol

One problem per worker; every attempt is bounded by wall clock only — no per-call caps. Budgets grew as the campaign matured: 20 min for the first probes (28–29 July), 30 min mid-campaign, and 60 min for the two full passes and all later runs. Architect arms additionally carry a refinement-iteration budget (5 for **river-vintage**, whose

per-attempt cutoff is already halved by the watcher variant; 8 for every other architect arm).

Failure triage distinguishes infrastructure failures from problem failures: catastrophic errors (quota, bridge down) pause the pass and *release* the claim without burning an attempt; problem-specific errors retry once, then mark the item *skipped*, which excludes it from the denominator rather than counting as a miss. A watchdog aborts streams wedged 5 minutes past the bridge’s own deadline — a stuck-socket backstop, not a prover cap. Every finished attempt files a research row (tagged `benchmark:fatex:<runId>`) into the same tables as production runs, with full per-attempt metrics.

3.5 Metrics

Pass rate is proved/scored, where scored = proved + unsolved + refuted (skipped excluded). Costs are the CLI-reported `total_cost_usd` summed per item; for Claude-driver runs this is a *notional API-equivalent* figure (the operator runs on a flat-rate subscription), while Grok-driver costs are real API spend. Times are per-item wall clock from claim to scoring.

4. Head-to-head: decomposition vs. the flat agent

The campaign’s central comparison is LUF-Anonymous (Ultra Fleeting: blueprint → parallel node provers → refinement) against LC-II-Irrationality (Control II: one continuous agent with verifier + search, no decomposition). Both ran claude-sonnet-5 [4], 60-minute budgets, over the same 98 items.

Table 1: Full-pass comparison. Solve time is over proved items only.

	Ultra Fleeting	Control II
Proved / scored	28 / 98	38 / 98
Pass rate	28.6%	38.8%
Solved by this arm only	2	12
Mean (median) solve time	827 s (562 s)	1,395 s (1,218 s)
Mean attempts per scored item	1.24	1.63
Median tool calls / item	163	110
Median LLM calls / item	233	212
Notional cost	\$825.53	\$878.57
Notional cost per solve	\$29.48	\$23.12
Verifier toolchain	v4.32.0	v4.29.1

4.1 Agreement and discordance

The two arms agree on 26 solves. Control II additionally solved 12 problems Ultra could not (`fatex_006`, `027`, `033`, `034`, `038`, `039`, `045`, `051`, `055`, `059`, `063`, `068`); Ultra’s unique contribution was 2 (`fatex_061`, `fatex_067`). Treating the 14 discordant problems as Bernoulli trials, an exact two-sided sign test gives $p = 212/2^{14} \approx 0.013$: the gap is unlikely to be noise, despite $n = 1$ passes per arm.

4.2 Interpretation

Three observations qualify the headline number rather than reverse it:

1. **When decomposition works, it is fast.** Ultra’s median solve took 562s against Control II’s 1,218s — parallel node-filling genuinely buys wall-clock speed on problems whose blueprints are right. Its per-run internals (median 6 nodes, 4 solved, 1 forfeited; median 2 blueprint iterations) show most solved problems needed only shallow decomposition.
2. **The failure mode is the blueprint, not the nodes.** On Ultra’s 70 misses the pipeline pays a fixed planning tax before any node prover runs, and a wrong skeleton caps what node provers can achieve. The flat agent spends the same minutes probing the compiler directly [26], and its higher attempt count (1.63 vs. 1.24) shows the budget being recycled into fresh whole-proof attempts — which FATE-X’s mid-range apparently rewards.
3. **Both arms die on the same wall.** In the top quartile the two arms converge (2/25 each, identical problems), so the architecture choice matters in the middle of the difficulty range, not at the top (§4.4).

Some FATE-X items are “flat-solvable but decomposition-hostile”: on `fatex_003` (a finite-index coset-representative theorem) both full passes succeeded, yet a proof forced through an up-front skeleton of `have`-holes creates sub-goals harder than the original (§5.1) — a concrete case where committing to structure early costs rather than helps.

4.3 Per-problem economy on shared solves

On the 26 problems *both* arms prove we can compare their resource use head-to-head, item by item (Fig. 1). The two are broadly comparable per problem: neither arm is systematically cheaper, faster, or more frugal with calls. What stands out are a few spikes where Control II pays heavily for a problem Ultra dispatches cheaply — `fatex_069` and `fatex_083` cost it \$13–14 and 30–40 min at 290–333 LLM calls, versus Ultra’s <\$3 and <10 min — and the mirror cases where Ultra pays more (`fatex_040`, `fatex_084`). The lesson: the aggregate cost/time gap of Table 1 is *not* a per-problem efficiency difference on shared work; it comes from the extra (harder) problems each arm uniquely solves and from budget burned on misses (§7).

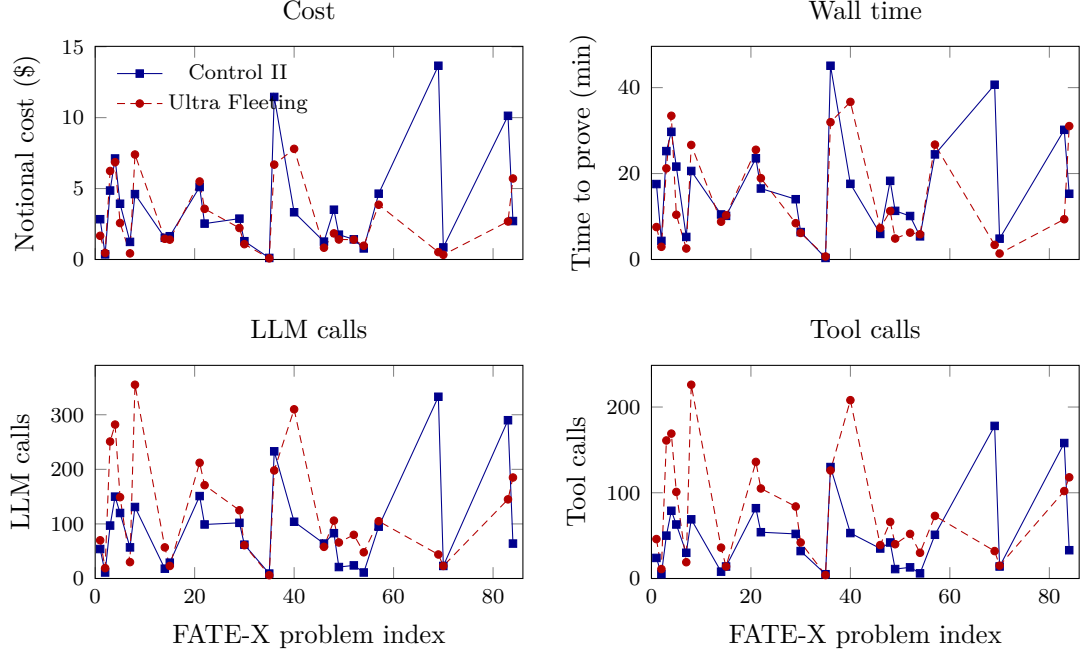


Figure 1: Per-problem economy on the 26 problems both arms prove, against problem index: notional cost, wall-clock solve time, LLM calls, and tool calls. Ultra Fleeting (dashed) tends to finish the easy shared items faster via parallel node-filling; Control II (solid) occasionally pays a large premium on a few (`fatex_069`, `fatex_083`). No arm dominates the shared set.

A sharper reading of Fig. 1 cuts against FATE-X’s headline framing. The benchmark is *ordered by increasing difficulty*, yet among the problems that were actually *solved*, none of the four cost measures trends upward with problem index: a solved item near rank 84 costs about what a solved item near rank 4 does. Difficulty, as FATE-X orders it, does *not* show up as a solved problem being more expensive to prove.

Where the ordering *does* hold is the *rate* of solving. Figure 2 plots cumulative solves against index for both arms: the curves climb almost one-for-one over the early items, bend through the middle, and go flat after the top quartile. The slope — solves per unit of index — decays monotonically, exactly what an increasing-difficulty ordering predicts. FATE-X’s difficulty is therefore expressed in *whether* a problem is solvable at all, not in *how hard* the solvable ones are to close.

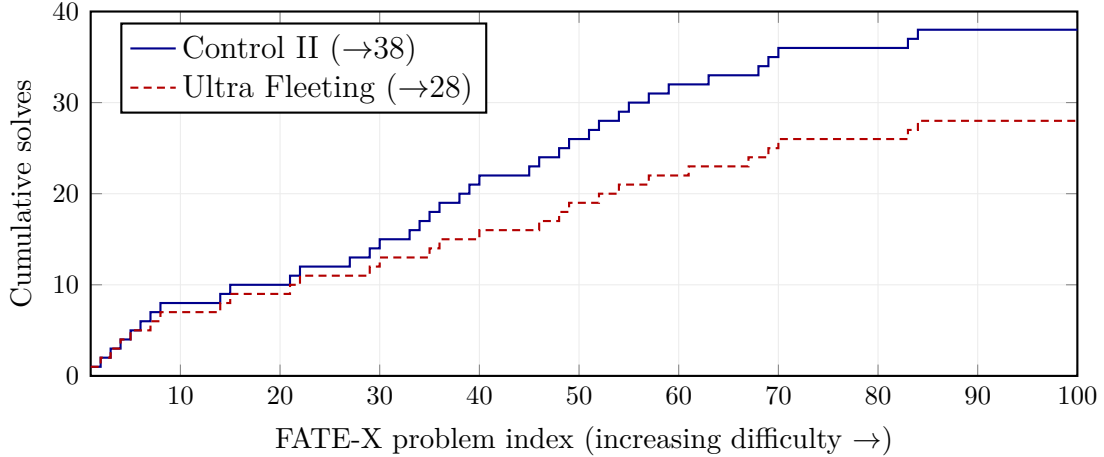


Figure 2: Cumulative solves against problem index. The slope — solves per unit index — falls monotonically and flattens after item 84: FATE-X’s increasing difficulty manifests as a declining *solve rate*, not as costlier proofs among the items that do get solved (contrast Fig. 1).

4.4 Difficulty gradient

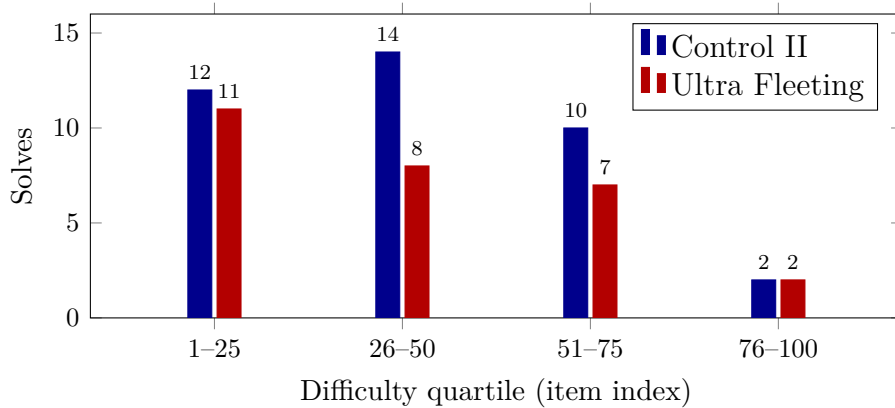


Figure 3: Solves by FATE-X difficulty quartile (24–25 scored items each; the benchmark is ordered by increasing difficulty [13]). Control II’s entire margin is the two middle bands; in the hardest quartile both arms solve the same two problems.

Control II’s entire margin comes from the middle bands (26–75: 24 vs. 15). The hardest-quartile solves are the same two problems for both arms (`fatex_083`, `fatex_084`), consistent with external reports that frontier provers solve single digits on upper FATE-X [7]. Whatever separates items 76–100 from the rest, it is not addressed by either architecture at a 60-minute budget.

4.5 Solves by topic area

FATE-X tags every item with a topic [13]. Table 2 breaks the two full passes down by the benchmark’s own finest tag (topics with $n \geq 4$ scored items shown; the remainder are in Appendix A).

Table 2: Solves by FATE-X topic tag, both full passes.

Topic	Scored	Ultra	Control II
Localization and Decomposition of Ideals	18	1	1
Galois theory	10	4	4
Ideals and Modules	8	3	5
Integral Dependence and the Nullstellensatz	7	2	3
Tensor Product and Flatness	7	0	2
Depth, Cohen–Macaulay and Gorenstein rings	7	1	2
Heights and Dimensions	6	3	2
Group Actions and Sylow theorems	5	3	4
Regular Sequences and Regular Local Rings	4	3	3
Algebraic and Arithmetic Dynamics	4	1	1

Two structural facts emerge. First, the benchmark’s largest topic — localization and primary decomposition, 18 items — is nearly untouched by both arms (1/18 each): this single topic accounts for over a quarter of all misses and is where FATE-X’s difficulty actually lives for us, more so than the nominal difficulty rank. Second, Control II’s margin is *broad*, not deep: +2 on Ideals and Modules, +2 on Tensor/Flatness, +1 across four further topics — there is no single subject the flat agent “understands” better, which is consistent with its advantage being mechanical (more whole-proof attempts per hour, §4) rather than mathematical. Ultra’s only topic-level win is Heights and Dimensions (3 vs. 2), the home of both of its unique solves (§5).

5. Case studies

5.1 fatex_003: flat-solvable, decomposition-hostile

The clearest single-problem signal in the campaign:

```
theorem exists_leftCoset_rightCoset_representative
  (G : Type) [Group G] (H : Subgroup G) [H.FiniteIndex] :
  ∃ S : Set G, Subgroup.IsComplement S H ∧ Subgroup.
    IsComplement H S := by
  sorry
```

(“A finite-index subgroup admits a common system of representatives for its left and right cosets.”) Both full passes solved it — Control II and Ultra’s whole-pipeline pass. The proof is a known short argument via Hall’s theorem-style matching already latent in Mathlib [24]; what it punishes is *premature structure*: a skeleton that commits to constructing S piecewise creates holes harder than the original goal. Ultra survives it because its blueprint stage may emit a near-trivial skeleton and otherwise fall back to a whole-proof attempt. This is the concrete instance behind the “decomposition-hostile” label used in this report.

5.2 fatex_061 and fatex_067: where decomposition wins

Ultra’s two unique solves are topical neighbours (both tagged Heights and Dimensions):

`formallySmooth_of_formallySmooth_quotient`
`isEmpty_mvPowerSeries_tensor_mvPowerSeries_algEquiv`

— formal smoothness descends along a square-zero quotient, and there is no k -algebra isomorphism $k[[x]] \otimes_k k[[y]] \cong k[[x, y]]$. Both have natural lemma decompositions — an ideal-power calculation and a cardinality/completeness obstruction respectively — and both defeated the flat agent’s iterate-against-the-compiler style within budget. They are the existence proof that the architect pipeline adds *some* capability; the finding of §4 is that this capability is currently narrow (2 problems) relative to its planning tax (10 problems).

5.3 fatex_083 and fatex_084: the only top-quartile solves

Both arms’ hardest-band solves are the same pair:

`IsRegularLocalRing.flat_local_of_regular`
`exists_directSum_free_free_of_projective`

— regularity descends along a flat local homomorphism, and Eilenberg’s swindle (for projective M there is a free N with $M \oplus N$ free). Both are textbook-named theorems whose informal proofs are certainly in the training distribution [4], and both have short Mathlib-adjacent formal skeletons. That the *same* two problems fall to two very different architectures suggests the top quartile is currently gated by statement-level familiarity, not by search strategy — consistent with the contamination caveat of §9 and with the flat difficulty profile both arms show above item 76.

6. The base agent alone (Control III)

A third full pass, LC-III-Await (Leak Control III), strips the agent to nothing: Claude Sonnet 5 is asked for a complete proof and told only whether each attempt passed a deterministic Leak IV gate — no error text, no search, no compiler feedback — carrying forward only its own past attempts and the attempt number. Over 42 scored items it proved 1 (`fatex_002`). We report it only in passing: the single thing it establishes is that the base agent, unaided, cannot make headway on FATE-X — the capability in the other two arms comes from tool access and whole-proof iteration, not raw model recall. No per-item obstacle was recorded, so we make no claim about *why* beyond the absence of tools and feedback.

7. Cost analysis

The two full passes cost \$825.53 (Ultra Fleeting) and \$878.57 (Control II) in CLI-reported `total_cost_usd`. These are *notional* API-equivalents — the runs were driven on Claude’s Max plan (20× usage gave comfortable headroom for parallel benchmarking), not metered API spend — so they measure compute intensity, not cash.

On that measure the flat agent is both cheaper per solve (\$23.12 vs. \$29.48) *and* more capable, so on FATE-X there is no cost–quality trade-off to argue about: Control II dominates on both axes.

Where the compute goes is the more useful signal. A failed item is not a cheap early exit — it thrashes against the compiler for the full budget, costing 3–4× the tool calls of a solved item in both arms (Fig. 4). Ultra’s 70 misses alone are ~70 budget-hours for zero score. Cheap early rejection of unpromising problems is therefore the highest-leverage efficiency work this comparison points to.

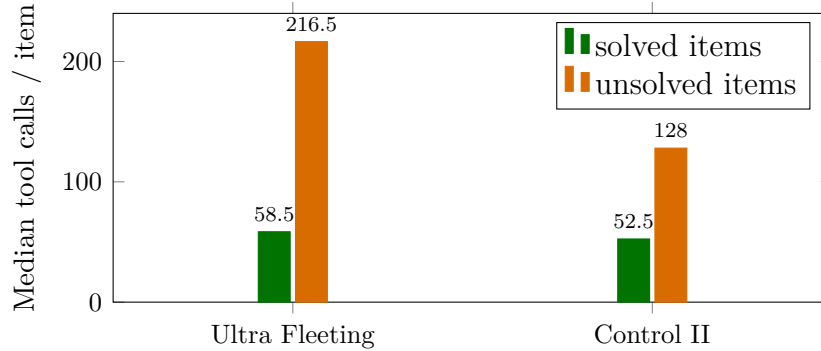


Figure 4: Where the compute goes: an *unsolved* item costs 3–4× the tool calls of a solved one in both arms. Failures thrash against the compiler for the full budget rather than exiting early, which is why misses dominate spend.

8. Operational incidents and data integrity

A benchmark is only as good as its bookkeeping. Three incidents from this campaign shaped the protocol and are documented here for reproducibility:

1. **Resume-path data loss (found 8 Aug, fixed commit f20f130).** The console’s “from here” requeue reset every non-running item at/after the target to *pending* — including already-scored ones. Because requeue paths never null the stored proof/metrics columns, the 19 wiped items of LUF-Anonymous (fatex_071–089: 2 proved, 17 unsolved) were recovered by restoring status from the preserved columns, after a full-run JSON backup. The fix restricts tail-requeue to pending/skipped items; the LUF-Anonymous figures throughout this report are post-recovery.
2. **Certificate wipe on failed re-verification (fixed).** A failed re-verify of an already-proven problem could clear its stored certificate before staging the new result, destroying a verified proof on infrastructure flake. Re-verification now stages before it clears.
3. **Verifier schema mismatch.** Leak IV’s verify endpoint takes {script} while Leak XIV takes {code, ...}; mis-wiring surfaces as what looks like a bridge bug and, worse, as a wrong-toolchain gate. Run manifests now log the armed verifier and toolchain line per run, which is how the v4.29.1/v4.32.0 split in §3.2 is known rather than assumed.

The triage rules of §3.4 (release vs. retry vs. skip) exist because of a fourth, older failure class: infrastructure flake silently consuming retry budget. Releasing a claim now decrements the attempt counter, so no quota outage can burn a problem’s attempts.

9. Threats to validity

1. **Single passes.** Each full-pass cell is $n=1$ per problem; the sign test in §4 conditions on discordant pairs but cannot correct for run-level luck (service latency, provider load).
2. **Sanitisation.** LUF benchmark run had a pre-sanitisation round to ensure theorems are in the shape LUF expects (everything compacted into one theorem signature, statements all elaborate), but this process was omitted for LC-II benchmark run. It is worth mentioning LC-II’s toolchain was one minor version ahead of the FATE-X pin.
3. **Toolchain confound.** Ultra verified against Mathlib v4.32.0, Control II against v4.29.1 (§3.2); FATE-X itself pins v4.28.0 [13]. Lemma renames and automation changes between Mathlib versions can shift item difficulty in either direction. The two excluded items are excluded from *both* arms, so the headline comparison is internally consistent, but absolute rates are not directly comparable with the FATE-X leaderboard without further restrictions for fairness. [7].
4. **Contamination.** FATE-X sources include textbooks and exam archives that plausibly appear in model training data, stated by the authors of the benchmark in their paper. [4]; formal statements are recent, informal ancestors may not be. This affects absolute rates, though it should affect both arms symmetrically.
5. **Notional costs.** Claude-driver dollar figures are API-equivalent estimates, not actual spend; for the reported benchmarks, we used Claude’s cheaper Max Plan (20x usage provided comfortable headroom for parallel benchmarking).

10. Conclusions and next steps

On FATE-X, at current budgets, a lifted decomposition scaffolding does not beat a well-tooled flat agent, despite the scaffolding augmenting proving capacity of small sized LLMs to SOTA levels: Control II leads Ultra Fleeting 38.8% to 28.6% on identical budgets and the same model ($p \approx 0.013$), at lower cost per solve (§4). Decomposition earns its keep only in speed (562s vs. 1,218s median per solve) and on exactly two problems the flat agent missed (§5). Both arms hit the same wall in the benchmark’s top quartile, and a base agent stripped of tools and feedback manages just 1/42 — so on this benchmark the capability comes from tool access and whole-proof iteration, not from the decomposition scaffold.

The comparison points to these follow-ups, in priority order:

1. **Blueprint-quality gate.** Ultra’s losses concentrate in planning: a cheap blueprint sanity check before committing node budget (cf. subgoal-quality filtering in [21]) is the pipeline’s most promising fix.
2. **Early rejection.** Failed items burn the full budget (§7); a cost estimator over proof-cost history that abandons unpromising problems early is the highest-leverage efficiency work.
3. **Toolchain-matched re-run.** Re-verify one full pass on the other verifier group to bound the v4.29.1/v4.32.0 confound directly.

4. **Top-quartile study.** Items 76–100 resist both architectures; qualitative failure analysis on `fatex_085–100` should precede any further architecture work aimed at them.

References

- [1] Leni Aniva et al. *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*. 2024. arXiv: [2410.16429](https://arxiv.org/abs/2410.16429).
- [2] Anonymous. *Submission 3bD19r4jqh: Subgoal Decomposition for Neural Theorem Proving*. <https://openreview.net/pdf?id=3bD19r4jqh>. Bibliographic details to be completed; OpenReview record was not machine-retrievable at time of writing. 2025.
- [3] Anthropic. *Building Agents with the Claude Agent SDK*. <https://www.anthropic.com/engineering/building-agents-with-the-claude-agent-sdk>. On the agent harness that wraps the model; see also the agent-terminology glossary at <https://huggingface.co/blog/agent-glossary>. 2025.
- [4] Anthropic. *Claude Sonnet 5*. <https://www.anthropic.com/news/claude-sonnet-5>. 2026.
- [5] Anthropic. *Introducing the Model Context Protocol*. <https://www.anthropic.com/news/model-context-protocol>. 2024.
- [6] Mikael Bashir. *Leak-II: A Ghost-Daemon Snapshot Layer over PyPantograph*. Latest copy of Leak-II, current as of the access date. Aug. 15, 2026. URL: <https://github.com/mikael-bashir/Leak-II> (visited on Aug. 15, 2026).
- [7] BenchmarkList. *FATE-X Benchmark Leaderboard*. https://benchmarklist.com/benchmarks/fate_x/. Accessed 2026-08-01. 2026.
- [8] Joachim Breitner. *Loogle: Expression Search for Mathlib*. <https://loogle.lean-lang.org/>. 2024.
- [9] CompeteMath. *CompeteMath Practice Arena*. <https://www.competemath.com/practice>. 2026.
- [10] CompeteMath. *What is Leak?* <https://www.competemath.com/about/leak>. 2026.
- [11] Kelly J. Davis. *Gödel’s Poetry*. Recursive proof decomposition over an AST-extended Kimina Lean server; reports 90.4% on miniF2F without decomposition. 2025. arXiv: [2512.14252](https://arxiv.org/abs/2512.14252).
- [12] Frenzy Math. *FATE-X release v4.28.0*. <https://github.com/frenzymath/FATE-X/releases/tag/v4.28.0>. Tagged release pinning Lean and Mathlib v4.28.0. 2025.
- [13] Frenzy Math. *FATE-X: Formal Algebra Theorem Evaluation — Extra Hard*. <https://github.com/frenzymath/FATE-X/tree/main/FATEX>. 100 Lean-formalised abstract- and commutative-algebra exercises intended to exceed PhD qualifying-exam difficulty and Mathlib coverage. 2025.
- [14] Goedel-Architect Team. *Goedel-Architect: Streamlining Formal Theorem Proving with Blueprint Generation and Refinement*. Blueprint dependency-graph pipeline with the DeepSeek-V4-Flash (284B-A13B) backbone. 2026. arXiv: [2606.06468](https://arxiv.org/abs/2606.06468).

- [15] Google DeepMind. *AI achieves silver-medal standard solving International Mathematical Olympiad problems*. <https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/>. 2024.
- [16] Hugging Face. *Spaces Overview*. <https://huggingface.co/docs/hub/en/spaces-overview>. 2026.
- [17] Guillaume Lample et al. “HyperTree Proof Search for Neural Theorem Proving”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2022.
- [18] Morph Labs. *Moogler: Semantic Search over Mathlib*. <https://www.moogler.ai/>. 2024.
- [19] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 Theorem Prover and Programming Language”. In: *Automated Deduction – CADE 28*. Springer, 2021.
- [20] Stanislas Polu and Ilya Sutskever. *Generative Language Modeling for Automated Theorem Proving*. arXiv:2009.03393. 2020.
- [21] Z. Z. Ren et al. *DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition*. 2025. arXiv: [2504.21801](https://arxiv.org/abs/2504.21801).
- [22] Austin Shen and Yunong Shi. *Keep the Proof State Live: Snapshotting for Efficient Tactic Search in Lean 4*. Reports elaboration overhead exceeding 99% of per-branch wall time, and 5.6–50× speedups from state reuse. 2026. arXiv: [2605.25556](https://arxiv.org/abs/2605.25556).
- [23] Stanford Centaur. *PyPantograph: A Python Interface to the Pantograph Lean 4 Interaction Engine*. <https://github.com/stanford-centaur/PyPantograph>. Accessed 2026-08-01. 2025.
- [24] The mathlib Community. “The Lean Mathematical Library”. In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. 2020.
- [25] George Tsoukalas et al. “PutnamBench: Evaluating Neural Theorem-Provers on the Putnam Mathematical Competition”. In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2024.
- [26] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023. arXiv: [2210.03629](https://arxiv.org/abs/2210.03629).
- [27] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. “miniF2F: a cross-system benchmark for formal Olympiad-level mathematics”. In: *International Conference on Learning Representations (ICLR)*. 2022.

A. Per-problem results

Full-pass outcome for every FATE-X item, in the benchmark’s difficulty order [13]. UF = LUF-Anonymous (Ultra Fleeting), C2 = LC-II-Irrationality (Control II); ✓ = proved, – = unsolved within budget. Items 9 and 41 do not elaborate under the campaign toolchains (§3.1) and were excluded. Theorem names are FATE-X’s own; topics are the benchmark’s finest tag.

Table 3: Per-problem outcomes for the two full passes.

#	Theorem	Topic	UF	C2
1	isPrincipalIdealRing_of_associated...	PID, ED and UFD	✓	✓
2	card_minimal_normal_subgroup_le_2	Subgroups and Quotient g...	✓	✓
3	exists_leftCoset_rightCoset_repres...	Subgroups and Quotient g...	✓	✓
4	add_one_eq_two_pow_of_sylow_subgro...	Group Actions and Sylow ...	✓	✓
5	maximal_abelian_normal_subgroup_of...	p-Groups, Nilpotent Grou...	✓	✓
6	not_isSimpleGroup_of_card_eq_396	Group Actions and Sylow ...	–	✓
7	not_isSimpleGroup_of_card_eq_1785	Group Actions and Sylow ...	✓	✓
8	quaternionAlgebra_isomorphic_to_ma...	Group Actions and Sylow ...	✓	✓
9	sylow_subgroup_not_normal_of_maxim...	Polynomials	excl.	
10	isPrincipalIdealRing_quot_X_pow_tw...	Algebraic Number Theory ...	–	–
11	not_isomorphic_euclideanDomain	Basic Definitions and Ex...	–	–
12	isPrincipalIdealRing_of_quadratic...	Integral Dependence and ...	–	–
13	sq_eq_self_of_not_unit	Galois theory	–	–
14	isomorphic_real_of_fractionRing_is...	Galois theory	✓	✓
15	exists_swap_stepwiseQuotient	Galois theory	✓	✓
16	normalClosure_isPEExtension_of_isPE...	Explicit Computations	–	–
17	countable_index_of_maximal_subfiel...	Galois theory	–	–
18	isEmpty_embedding_intermediateFiel...	Galois theory	–	–
19	galoisGroup_iso_quaternion_group	Algebraic Number Theory ...	–	–
20	generated_single_elem_of_degree_le_p	Galois theory	–	–
21	intermediateField_rank_eq_ringChar	Galois theory	✓	✓
22	finite_algebraic_integers_of_finit...	Galois theory	✓	✓
23	ratio_tendsto_one_of_irreducible	Galois theory	–	–
24	galoisGroup_iso_of_distinct_primes	Galois theory	–	–
25	fixedField_eq_algebra_adjoin	Integral Dependence and ...	–	–
26	card_one_or_two_of_finite_closed_s...	Integral Dependence and ...	–	–
27	isGalois_and_rank_eq_of_isPrimitiv...	Integral Dependence and ...	–	✓
28	infinite_conj_of_ne_1_absoluteGalo...	Localization and Decompo...	–	–
29	isClosed_zpowers_iff_isOfFinOrder	Localization and Decompo...	✓	✓
30	integrallyClosedIn_of_complement_m...	Integral Dependence and ...	✓	✓
31	UFD_of_ge_5	Localization and Decompo...	–	–
32	UFD_of_adicCompletion_UFD	Localization and Decompo...	–	–
33	isNoetherianRing_of_fg_of_isNoethe...	PID, ED and UFD	–	✓
34	powerSeries_not_integrallyClosed_o...	Tensor Product and Flatn...	–	✓
35	noetherian_of_prime_ideals_fg	Ideals and Modules	✓	✓
36	associatedPrimes_hom_eq_support_in...	Integral Dependence and ...	✓	✓
37	ufd_quotDetSubOne	Integral Dependence and ...	–	–
38	free_dualNumber_and_rank_eq_2	Tensor Product and Flatn...	–	✓
39	finite_primes_lies_over_of_finite...	Heights and Dimensions	–	✓
40	free_of_rank_iff	Regular Sequences and Re...	✓	✓
41	isNoetherianRing_and_krullDim_eq_top	Tensor Product and Flatn...	excl.	
42	nonEmpty_ringEquiv_of_sub_in_cube	Tensor Product and Flatn...	–	–
43	IsRegularLocalRing.frobenius_flat	Ideals and Modules	–	–
44	quot_x2_sub_y2_y2_sub_z2_xy_yz_zx...	Completions and Hensel’s...	–	–

#	Theorem	Topic	UF	C2
45	free_of_free_resolution	Completions and Hensel's...	–	✓
46	module_flat_iff	Ideals and Modules	✓	✓
47	isEmpty_isomorphism_UFD_of_quotient	Depth, Cohen-Macaulay Ri...	–	–
48	isAbsolutelyFlat_iff_principal_ide...	Smoothness and the Modul...	✓	✓
49	principal_ideal_idempotent_iff_fg_...	Ideals and Modules	✓	✓
50	isNoetherianRing_of_isLocalRing_of...	Tensor Product and Flatn...	–	–
51	adicCompletion_faithfullyFlat_iff	Polynomials	–	✓
52	generate_unit_ideal_of_quotient	Regular Sequences and Re...	✓	✓
53	isGorensteinRing_quot_x2_sub_y2_y2...	Tensor Product and Flatn...	–	–
54	not_zero_divisor_of_hausdorff_of_d...	Basic Definitions and Ex...	✓	✓
55	stablyFree_iff_free_of_not_fg	Ideals and Modules	–	✓
56	projective_of_faithfullyFlat_base_...	Smoothness and the Modul...	–	–
57	completely_integrally_closed_polyn...	Dedekind Domains and DVRs	✓	✓
58	flat_of_flat_over_quotient	Homological Methods	–	–
59	exists_unique_valuation_eq	Polynomials	–	✓
60	invertible_iff_codimension_one	Depth, Cohen-Macaulay Ri...	–	–
61	formallySmooth_of_formallySmooth_q...	Heights and Dimensions	✓	–
62	adicCompletion_equiv_of_smooth	Tensor Product and Flatn...	–	–
63	surjection_of_formally_unramified	Ideals and Modules	–	✓
64	homogeneous_coordinate_ring_not_is...	Depth, Cohen-Macaulay Ri...	–	–
65	Polynomial.isGorensteinRing	Homological Methods	–	–
66	exists_eq_ofList_and_isRegular_of_...	Depth, Cohen-Macaulay Ri...	–	–
67	isEmpty_mvPowerSeries_tensor_mvPow...	Heights and Dimensions	✓	–
68	localization_jacobson_of_one_lt_ri...	Depth, Cohen-Macaulay Ri...	–	✓
69	IsRegularLocalRing.regularAtPrime	Depth, Cohen-Macaulay Ri...	✓	✓
70	exists_minimalPrime_map_zero	Regular Sequences and Re...	✓	✓
71	fixedPoints_isCohenMacaulayRing	Regular Sequences and Re...	–	–
72	isCohenMacaulay_extendScalars_over...	Completions and Hensel's...	–	–
73	mvPolynomial_quotient_isCohenMacau...	Tensor Product and Flatn...	–	–
74	IsRegularLocalRing.gorensteinAtReg...	Depth, Cohen-Macaulay Ri...	–	–
75	gradedAlgebra_isCohenMacaulay_iff_...	Ideals and Modules	–	–
76	IsCatenary.of_noetherian_ufd_of_di...	Heights and Dimensions	–	–
77	infinite_intermediate_primes	Ideals and Modules	–	–
78	IsCohenMacaulayLocalRing.isGorenst...	Localization and Decompo...	–	–
79	IsLocalRing.not_isCompleteIntersec...	Group Actions and Sylow ...	–	–
80	isCohenMacaulayRing_of_dimension_t...	Polynomials	–	–
81	generated_by_regular_sequence_iff	Homological Methods	–	–
82	subring_iso_mvPowerSeries_over_DVR	Heights and Dimensions	–	–
83	IsRegularLocalRing.flat_local_of_r...	Heights and Dimensions	✓	✓
84	exists_directSum_free_free_of_proj...	Algebraic and Arithmetic...	✓	✓
85	exist_euclideanDomain_not_norm_nat	Algebraic and Arithmetic...	–	–
86	dimension_convex	Algebraic and Arithmetic...	–	–
87	exists_polynomial_ringEquiv_isEmpt...	Algebraic and Arithmetic...	–	–
88	quotient_not_UFD	Localization and Decompo...	–	–
89	not_isSimpleGroup_of_card_eq_336	Localization and Decompo...	–	–
90	not_finiteType_inf_algebraMap_range	Localization and Decompo...	–	–
91	pic_three_lines	Localization and Decompo...	–	–
92	dimension_sequences_of_one_dimensi...	Localization and Decompo...	–	–
93	exists_integral_finiteType_not_fin...	Localization and Decompo...	–	–
94	zeroSet_finite_or_contain_arithmet...	Localization and Decompo...	–	–
95	p_map_ne_p	Localization and Decompo...	–	–
96	ratFunc_square_is_poly_of_orbit_co...	Localization and Decompo...	–	–
97	exists_point_not_in_zero_set	Localization and Decompo...	–	–
98	exists_maximal_ideal_not_in_finite...	Localization and Decompo...	–	–
99	equiv_of_aut_equiv	Localization and Decompo...	–	–

#	Theorem	Topic	UF	C2
100	<code>free_of_countably_generated_projec...</code>	Localization and Decompo...	–	–